

VOLUME 6

COMPLETE VOLUME — CHAPTERS 6.1–6.9

Mobile App Architecture

The Flutter App, Fully Specified and Ready to Build

Document Title	Volume 6 — Mobile App Architecture (Combined)
Volume Reference	Volume 6
Chapters Included	6.1 through 6.9 (9 chapters)
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Governed By	Volume 0 — Project Foundation; grounded in Volumes 1, 3, 4, and 5

Table of Contents

This volume turns Volume 3's architecture into a buildable Flutter app: the bootstrap sequence, the literal project structure, the concrete routing table, dependency injection in practice, the finalized local database schema, preferences, secure token storage, notification handling, and global error handling.

Chapter	Title
6.1	Flutter Architecture
6.2	Project Structure
6.3	Routing
6.4	Dependency Injection
6.5	Local Database
6.6	Preferences
6.7	Secure Storage
6.8	Notification Architecture
6.9	Error Handling

VOLUME 6 — MOBILE APP ARCHITECTURE

CHAPTER 6.1

Flutter Architecture

Turning Volume 3's Blueprint Into an Implementation-Ready Design

Document Title	Flutter Architecture
Chapter Reference	6.1
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Builds On	Volume 3, Chapters 3.4–3.6 (Component/Module Architecture, Folder Structure)

1. Purpose

Volume 3 fixed the shape: five layers (Presentation → State → Domain → Data → Platform Services), eight feature modules, and the folder tree. Volume 6 is where that shape becomes buildable — the exact wiring between `app.dart`, `ProviderScope`, `go_router`, and the first screen the Collector or Admin sees. Nothing here contradicts Volume 3; this chapter is its execution detail.

2. App Bootstrap Sequence

```
main() →
  1. WidgetsFlutterBinding.ensureInitialized()
  2. Initialize Firebase (Ch.6.7's token flow, Ch.6.8's FCM depend on this)
  3. Open the local Drift database (Ch.6.5) — before any provider reads it
  4. runApp(ProviderScope(child: HumanArchiveApp()))

HumanArchiveApp (app.dart) →
  MaterialApp.router(routerConfig: goRouter) — Ch.6.3
```

3. What Volume 6's Remaining Chapters Fix

- 6.2 — the literal starter repo, beyond Volume 3.6's conceptual tree.
- 6.3 — the concrete `go_router` route table for every screen in Chapter 2.5's inventory.
- 6.4 — how Riverpod providers are actually organized and overridden, in practice.
- 6.5 — the exact Drift table definitions Volume 5.8 named but didn't fully specify.
- 6.6/6.7 — where non-sensitive preferences and sensitive tokens each live, and why they're different mechanisms.
- 6.8 — how Firebase Cloud Messaging (confirmed, Volume 4.1) actually reaches the app.
- 6.9 — what happens when any layer throws.

4. One Architectural Reminder Carried Forward

Layer direction still applies: nothing in this volume introduces a widget that talks to Drift directly, or a Repository that imports a Notifier — Volume 3, Chapter 3.4's inward-only dependency rule is the constitution this entire volume implements against, checked by the same import-boundary lint rules from Chapter 3.7.

Project Structure

The Literal Starter Repository

Document Title	Project Structure
Chapter Reference	6.2
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Extends	Volume 3, Chapter 3.6 (Folder Structure)

1. What's New Beyond Volume 3.6

Chapter 3.6 fixed `lib/`, `test/`, and the top-level layout conceptually. This chapter adds the files a real, freshly-cloned repository needs that weren't yet in scope for an architecture document — config files, environment scaffolding, and CI entry points, each cross-referenced to the Volume 7 chapter that configures it.

2. Additions to the Root

```
human_archive_app/
├── .fvmrc                # pinned Flutter SDK version (Ch.3.8; Volume 7.5)
├── .env.example          # documents required vars, no real secrets (Ch.6's
│                         # equivalent doesn't exist yet — see Volume 7.10)
├── .gitignore            # excludes build/, .dart_tool/ — NOT .g.dart (Ch.3.6)
├── .github/workflows/    # CI entry points (Volume 7.6/7.13)
├── melos.yaml            # not used — single package, per Ch.3.8, Section 2
└── (lib/, test/, android/, ios/, assets/ — unchanged from Ch.3.6)
```

3. Why .g.dart Files Are Committed, Not Ignored

A common Flutter default is to `.gitignore` generated files. This project deliberately does not: Chapter 3.6, Section 5 already fixed that generated files (`riverpod_generator`, `freezed`, `json_serializable` output) are committed, so that a fresh clone builds and runs correctly even before anyone remembers to run `build_runner` — a direct contributor to Volume 7's goal of a reproducible environment.

4. Native Project Folders

- `android/` carries the foreground-service manifest entries and notification channel setup (Volume 5.11, Volume 6.8) — configured once, not touched per feature.
- `ios/` carries the background `URLSession` and push notification entitlements (Volume 5.11, Volume 6.8) — built and signed via the cloud Mac service (Volume 7.7/7.8's Firebase/AWS setup feeds credentials into this build, not the other way around).

5. What This Chapter Defers

- Exact CI workflow YAML contents — Volume 7.
- Exact environment variable names/values — Volume 7.10.

CHAPTER 6.3

Routing

The Concrete go_router Table Behind Chapter 2.4's Navigation Model

Document Title	Routing
Chapter Reference	6.3
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	ADR-005 (Volume 3); Volume 2, Chapter 2.4 (Navigation Structure)

1. Top-Level Shell Routes

Path	Screen (Ch.2.5 ID)	Notes
/login	SH-02	No shell — full screen.
/collector	Collector tab shell (Ch.2.4 §2)	StatefulShellRoute with 5 branches: dashboard, projects, record, sessions, settings.
/admin	Admin tab shell (Ch.2.4 §3)	StatefulShellRoute with 4 branches: dashboard, projects, sessions, settings.

2. Collector Routes

Path	Screen (Ch.2.5 ID)	Notes
/collector/dashboard	C-03	Tab root.
/collector/projects	C-04	Tab root.
/collector/projects/:projectId/tasks/:taskId	C-06	Stack push from C-04/C-05.
/collector/checklist/:taskId	C-07 / C-08	Presented as a full-screen modal route, not a tab push (Ch.2.4).
/collector/record/:sessionId	C-09	Chrome-free — see Section 4.
/collector/sessions	C-11 / C-12	Tab root; C-12 is a sub-state of the same route, not a separate path.
/collector/settings	C-14	Tab root.

3. Admin Routes

Path	Screen (Ch.2.5 ID)	Notes
/admin/dashboard	A-01	Tab root.
/admin/projects	A-02	Tab root.
/admin/projects/:id	A-03	Stack push.
/admin/projects/:id/tasks/new	A-05 (create)	Modal route.
/admin/tasks/:id/assign	A-06	Modal route.
/admin/sessions	A-07	Tab root.
/admin/chunks/:chunkId/metadata	A-08	Modal overlay on A-07 (Ch.2.4 §3).

4. Route Guards

Auth guard: every route except /login redirects to /login if AuthState (Volume 3, Chapter 3.9) is unauthenticated — implemented once, in go_router's top-level redirect callback, not per-route.

Role guard: the same redirect callback sends an authenticated Collector attempting /admin/* (or vice versa) back to their own root — mirrors Chapter 2.4 §4's Role Router, now expressed as routing logic instead of a one-time screen.

Chrome-free override: the /collector/record/:sessionId route disables the system back gesture at the route level (a route-specific PopScope/onPopInvoked override) — the one deliberate exception already flagged in Volume 3, Chapters 3.2 and 3.9.

5. Deep Links (Reserved, Not Built for MVP)

go_router's URL-based routing (ADR-005's stated benefit) means a future deep link — e.g. opening a specific Task from a push notification (Chapter 6.8) — is a route the app already has, not a new navigation mechanism to build later. Not wired to any external entry point yet, since Volume 1 scoped notifications to Phase 2.

VOLUME 6 — MOBILE APP ARCHITECTURE

CHAPTER 6.4

Dependency Injection

How the Riverpod Provider Graph Is Actually Organized

Document Title	Dependency Injection
Chapter Reference	6.4
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	ADR-006 (Volume 3, Chapter 3.2)

1. Provider Placement Rule

Every provider lives in the same feature module folder as what it provides (Volume 3, Chapter 3.5) — a repository provider sits in that feature's data/ folder, a Notifier in its state/ folder. There is no central providers.dart file collecting every provider in one place, since that would recreate exactly the cross-module coupling Chapter 3.5's module boundaries exist to prevent.

2. Provider Layering, Worked Example

```
// features/recording/data/recording_repository.dart
@riverpod
RecordingRepository recordingRepository(RecordingRepositoryRef ref) {
  final db = ref.watch(driftDatabaseProvider);    // from core/
  final camera = ref.watch(cameraServiceProvider); // platform/
  return RecordingRepositoryImpl(db, camera);
}

// features/recording/state/recording_notifier.dart
@riverpod
class RecordingNotifier extends _$RecordingNotifier {
  RecordingState build() => const RecordingState.idle();
  Future<void> stop() async {
    final repo = ref.read(recordingRepositoryProvider); // reads, doesn't rebuild on
    repo change
    ...
  }
}
```

ref.watch is used when a provider's own output should rebuild if its dependency changes; ref.read is used inside methods that only need a one-time value at the moment of the call — this distinction is enforced in code review (Volume 3, Chapter 3.7's checklist) since misusing watch inside a method body is a common, subtle Riverpod bug.

3. Core-Level Providers

- driftDatabaseProvider, dioClientProvider, secureStorageProvider, and goRouterProvider all live in core/ (Volume 3, Chapter 3.5) — every feature module depends on core, so these are the only providers with genuinely app-wide scope.
- Firebase-derived providers (authStateProvider, fcmTokenProvider) also live in core/, since auth state gates every feature module's own behavior (Chapter 6.3's route guards read it directly).

4. Testing Overrides

```
final container = ProviderContainer(overrides: [
  recordingRepositoryProvider.overrideWithValue(FakeRecordingRepository()),
]);
```

This is ADR-006's stated payoff realized in practice — every repository provider is designed to be overridden this way, so Volume 9's test suite never needs a real Drift database, a real camera, or a real network call to test a Notifier's logic.

5. What This Chapter Defers

- The actual Drift schema behind `driftDatabaseProvider` — Chapter 6.5, immediately following.

VOLUME 6 — MOBILE APP ARCHITECTURE

CHAPTER 6.5

Local Database

The Exact Drift Schema Volume 5.8 Named but Didn't Fully Specify

Document Title	Local Database
Chapter Reference	6.5
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Resolves	Volume 0, Ch.0.2 — "local embedded database, finalized in Volume 6.5"

1. Drift Table Definitions

```
class LocalSessions extends Table {
    TextColumn get id => text();
    TextColumn get taskId => text();
    DateTimeColumn get startedAt => dateTime();
    TextColumn get status => text(); // 'in_progress' | 'complete'
    @override Set<Column> get primaryKey => {id};
}

class LocalChunks extends Table {
    TextColumn get id => text();
    TextColumn get sessionId => text();
    IntColumn get sequenceIndex => integer();
    TextColumn get localFilePath => text();
    TextColumn get s3ObjectKey => text();
    TextColumn get status => text(); // queued|uploading|failed|complete
    TextColumn get checksumSha256 => text();
    IntColumn get fileSizeBytes => integer();
    DateTimeColumn get localDeletedAt => dateTime().nullable();
    @override Set<Column> get primaryKey => {id};
}

class LocalChunkMetadata extends Table {
    TextColumn get chunkId => text(); // 1:1, references LocalChunks.id
    TextColumn get metadataJson => text(); // full Ch.4.5 JSON shape
    DateTimeColumn get verifiedAt => dateTime().nullable();
    @override Set<Column> get primaryKey => {chunkId};
}
```

2. Design Notes

metadata_json as a single column, not exploded fields: unlike the backend's chunk_metadata table (Volume 4, Chapter 4.4), the local mirror stores the whole Chapter 4.5 JSON object as one column — the device never needs to query individual metadata fields relationally (it only ever reads/writes the whole object), so exploding it into columns would add migration overhead for no query benefit.

No org_id / project_id locally: the device only ever holds its own Collector's data (BR-19), so org-level scoping — meaningful on the multi-tenant backend — has no purpose in a single-user local database.

3. Reactive Queries (Feeding Chapter 3.9's StreamNotifiers)

```
Stream<List<LocalChunk>> watchQueue() =>
    (select(localChunks)..where((c) => c.status.isNotValue('complete')))
        .watch(); // Drift's native reactive query — Ch.3.9's UploadQueue
                // StreamNotifier subscribes directly to this.
```

4. Migrations

Drift's schema versioning (ADR-002, Volume 3) is used from v1 onward — every future field addition (e.g. a new metadata group) ships as a numbered migration with an explicit onUpgrade step, never a destructive drop-and-recreate, since a Collector's queued-but-not-yet-uploaded chunks must survive an app update untouched.

5. What This Chapter Defers

- Non-sensitive user preferences (theme, upload mode) — Chapter 6.6 — deliberately kept out of Drift, using a lighter-weight mechanism instead.
- Auth tokens — Chapter 6.7 — deliberately never stored in Drift at all.

VOLUME 6 — MOBILE APP ARCHITECTURE

CHAPTER 6.6

Preferences

Small, Non-Sensitive, Per-Device Settings

Document Title	Preferences
Chapter Reference	6.6
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Why Not Drift

Chapter 6.5's Drift database exists for structured, relational, queryable data. Preferences are none of those — a handful of independent key-value flags that are read constantly (e.g. on every screen build) and never joined against anything. A lightweight preferences store avoids paying SQL query overhead for what is really just app-wide configuration.

2. Mechanism

`shared_preferences` (backed by `UserDefaults` on iOS, `SharedPreferences` on Android), wrapped behind a single `PreferencesRepository` so no other layer imports the package directly (Volume 3, Chapter 3.4's repository pattern applies here too, even though it isn't the network/Drift kind).

3. Stored Preferences

Key	Type	Purpose
<code>upload_mode</code>	enum: 'automatic' 'manual'	FR-UPL-02 — per-device configuration, not per-user (so a shared device can be configured independently of who's logged in).
<code>theme_mode</code>	enum: 'light' 'dark' 'system'	Chapter 2.8's Design System theme toggle, made persistent rather than resetting every app launch.
<code>onboarding_seen</code>	bool	Whether C-01's permission-priming carousel has already been shown on this device.
<code>last_synced_project_ids</code>	list<string>	Powers the offline-cache freshness check for C-04/A-02 (Volume 5, Chapter 5.8's <code>local_task_cache</code>).

4. Access Pattern

Read: exposed as a plain Riverpod provider per key (e.g. `uploadModeProvider`) rather than one giant "preferences" object — so a widget depending on `theme_mode` doesn't rebuild when `upload_mode` changes.

Write: always goes through `PreferencesRepository`'s typed setter methods, never a raw string key from calling code — preventing an easy-to-make typo in a key name from silently creating a second, disconnected preference.

5. What Never Lives Here

- Anything used for authorization or identity (auth tokens, role) — Chapter 6.7, a fundamentally different security posture.
- Anything that must survive a full app reinstall for legal/audit reasons — that's server-side data (Volume 4), never local preferences.

VOLUME 6 — MOBILE APP ARCHITECTURE

CHAPTER 6.7

Secure Storage

Where Tokens Live, and Why That's a Different Mechanism Than Chapter 6.6

Document Title	Secure Storage
Chapter Reference	6.7
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	NFR-SEC-03 (Volume 1); the token flow in Volume 4, Chapter 4.7

1. Mechanism

flutter_secure_storage, which persists via Android Keystore and iOS Keychain — hardware-backed, encrypted storage, categorically different from Chapter 6.6's plain-text preferences file. Anything that would cause harm if read off a lost or seized device belongs here, never in shared_preferences or a Drift column.

2. What's Stored

- The Firebase refresh token (the ID token itself is short-lived and kept only in memory via AuthState, Volume 3 Chapter 3.9 — never persisted at all, since it's trivially reissued from the refresh token).
- Nothing else, by design — Volume 1's metadata and Volume 4's business data never touch the device's secure storage; they live in Drift (Chapter 6.5) or the backend (Volume 4), neither of which is a secrets store.

3. Access Pattern

Wrapped behind SecureStorageService (Volume 3, Chapter 3.4's platform-services layer): the AuthRepository is the only caller — no feature module reads secure storage directly, matching the same single-entry-point discipline as every other platform service.

Read timing: only at app cold-start, to attempt silent re-authentication before falling back to the Login screen (SH-02) — not read repeatedly during normal use, since the in-memory AuthState is the source of truth once a session is established.

4. Why This Matters for NFR-SEC-03

NFR-SEC-03 treats locally stored video chunks as sensitive until upload is confirmed — that's a Chapter 6.5/Volume 5.8 concern (the raw file itself). This chapter closes the adjacent gap: the credential that could be used to impersonate a Collector or Admin against the backend (Volume 4, Chapter 4.8's authorization) is protected at least as strongly as the data it guards access to, using the platform's actual hardware-backed keystore rather than an ordinary file.

5. What This Chapter Defers

- Server-side token issuance and expiry policy — Volume 4, Chapter 4.7 (already specified there; this chapter only covers the on-device half).

VOLUME 6 — MOBILE APP ARCHITECTURE

CHAPTER 6.8

Notification Architecture

How Firebase Cloud Messaging Reaches the App in Every State

Document Title	Notification Architecture
Chapter Reference	6.8
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Provider	Firebase Cloud Messaging — confirmed, Volume 4, Chapter 4.1

1. Scope

Volume 1, Chapter 1.10 scoped Collector-facing Notifications (C-15) to Phase 2 — this chapter still specifies the architecture now, since Chapter 5.11's Android foreground-service notification (upload progress) is already an MVP need and shares the same underlying notification-channel setup as the future push notifications will.

2. Token Registration

- On successful login (Chapter 4.7), the app requests an FCM token and sends it to POST /v1/users/me/fcm-token (a small addition to Volume 4, Chapter 4.6's catalog) so the backend can target this specific device.
- A token refresh (FCM occasionally rotates tokens) is re-registered automatically via FCM's onTokenRefresh callback — no user-visible action.

3. Notification Channels (Android)

Channel	Purpose	Priority
upload_progress	The Chapter 5.11 foreground-service notification — MVP, always active during upload.	Low (silent, ongoing).
upload_alerts	Phase 2 — a chunk permanently failed and needs manual attention (C-15).	Default (sound/vibration).
assignments	Phase 2 — an Admin assigned a new Task (US-15-adjacent future story).	Default.

4. Handling by App State

Foreground: FCM's onMessage stream is handled in-app — Phase 2 messages render as an in-app banner rather than a system notification, since the Collector is already looking at the screen.

Background: the OS displays the system notification directly from the FCM payload; tapping it deep-links via Chapter 6.3's go_router routes (e.g. straight to the relevant Task).

Terminated: the same deep-link path is read from the launch payload when the app cold-starts from a notification tap — go_router's URL-based routing (ADR-005) makes this the same code path as a warm-start deep link, not a special case.

5. Relationship to Chapter 5.11's Upload Notification

The Android foreground-service notification is a local notification the app itself posts (not FCM-delivered), but it uses the same platform notification-channel infrastructure set up in this chapter — Volume 6 configures notification channels once, and both the MVP upload-progress case and the Phase 2 FCM case draw from that shared setup.

6. What This Chapter Defers

- The exact Phase 2 notification content/copy — a Volume 1-style requirements exercise revisited when Phase 2 begins, not fixed here.

VOLUME 6 — MOBILE APP ARCHITECTURE

CHAPTER 6.9

Error Handling

What Happens When a Layer Throws Something Nobody Expected

Document Title	Error Handling
Chapter Reference	6.9
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Builds On	Volume 3, Chapter 3.9, Section 5 (Error Modeling)

1. Two Kinds of Error, Two Different Handlers

Expected, modeled failures: the sealed failure types from Volume 3, Chapter 3.9, Section 5 (e.g. `ChecklistFailure`) — these are caught and pattern-matched at the Presentation layer into the exact Chapter 2.9 copy, and never reach a global handler at all.

Unexpected, uncaught exceptions: anything a layer didn't anticipate (a null the type system should have prevented, a plugin throwing an undocumented platform exception) — these are what this chapter's global handling exists for.

2. Global Handlers

- `FlutterError.onError` — catches framework/widget-layer errors (e.g. a rendering exception) and routes them to the crash reporter (Section 3) instead of only printing to console.
- `PlatformDispatcher.instance.onError` — catches errors outside the Flutter framework's own zone (async errors in platform channel callbacks, background isolates).
- Both handlers funnel into one `ErrorReportingService` (Volume 3, Chapter 3.4's platform-services layer) so there is exactly one place that decides what happens next, not two independent logging paths.

3. ADR-010 — Crash Reporting Tool

ADR-010 — Crash Reporting / Diagnostics Tool

Status: Accepted

Context:

Uncaught exceptions in the field (a Collector's device, hours from the nearest developer) are otherwise invisible; the team needs a way to see them without asking a Collector to describe a crash after the fact.

Options Considered:

- Sentry — strong Flutter support, but introduces a fourth external service alongside AWS and Firebase.
- A custom logging-to-backend endpoint — full control, but reinvents session grouping, stack symbolication, and alerting that a dedicated tool already solves well.
- Firebase Crashlytics — Firebase is already in the stack for Auth/FCM (Volume 4.1), so this adds zero new vendor relationships.

Decision:

Chosen: Firebase Crashlytics.

Consequences:

- Crash/error reports are automatically tied to the same Firebase project already used for Auth and FCM — one fewer dashboard to check.
- Crashlytics' non-fatal error logging is used for the modeled-but-still-noteworthy cases (e.g. a chunk hitting its final retry attempt, Volume 5, Chapter 5.13) so the team can see patterns in the field even when nothing technically 'crashed'.

4. What Never Goes Into a Crash Report

- Anything from Chapter 6.7's secure storage (tokens) or raw chunk_metadata GPS/device fields beyond what's needed to reproduce the bug — crash reports are scrubbed of anything Volume 8 (Security & Compliance) would classify as sensitive before being sent.

5. What This Chapter Defers

- Alerting thresholds and on-call response process for production crashes — Volume 8 and the engineering:incident-response process, not a mobile-architecture concern.